

A CAPSTONE PROJECT REPORT ON

REPAIRGRAPH

AI-Assisted Device Repairability Assessment, Economic Decision Support & Competitive Repair Marketplace Platform

Submitted in partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted By:	Under the Guidance of:
NAIDU RESHMANTH SAI	[Faculty Guide Name / Title]
Register No: 25BCE1112	[Designation / Department]
Department of Computer Science and Engineering	[Institution / University Name]
Academic Year: 2025–2026	Production URL: https://repairgraph.vercel.app

CERTIFICATE OF ORIGINAL WORK

This is to certify that the project report titled "REPAIRGRAPH: AI-Assisted Device Repairability Assessment, Economic Decision Support & Competitive Repair Marketplace Platform" submitted by NAIDU RESHMANTH SAI (Register Number: 25BCE1112) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2025–2026 is an authentic record of original work carried out under my supervision.

To the best of our knowledge and verified evaluation, the results, algorithms, architectures, and findings presented in this report have been independently implemented, automatedly tested across 82 validation test cases, and functionally validated in production cloud deployment. This work has not formed the basis for the award of any other degree, diploma, or fellowship elsewhere.

[Signature of Project Guide] [Guide Name & Designation] Department of Computer Science and Engineering	[Signature of Head of Department] [HOD Name & Designation] Department of Computer Science and Engineering
Date: _____ Place: _____	Internal / External Examiner: Signature: _____

CANDIDATE DECLARATION

I, NAIDU RESHMANTH SAI (Register Number: 25BCE1112), hereby declare that the capstone project report titled "REPAIRGRAPH: AI-Assisted Device Repairability Assessment, Economic Decision Support & Competitive Repair Marketplace Platform" is an authentic presentation of work carried out by me under academic supervision.

I confirm that all software code, full-stack Next.js architecture, Prisma relational models, deterministic decision formulas, test suites, and production cloud infrastructure on Vercel and Neon PostgreSQL described in this report represent true, implemented, and verified engineering work. The external libraries, official specifications, and statutory frameworks (including India's Right to Repair Portal and the E-Waste Management Rules, 2022) have been appropriately referenced.

NAIDU RESHMANTH SAI Register Number: 25BCE1112 Department of Computer Science and Engineering
Date: _____ Place: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my Project Guide and Faculty Advisor for their continuous guidance, technical insights, and encouragement throughout the ideation, system design, implementation, and cloud validation of RepairGraph.

I extend my appreciation to the Head of the Department and faculty members of the Department of Computer Science and Engineering for providing the academic infrastructure, computing facilities, and rigorous evaluation milestones that helped elevate this project into a production-grade software system.

Finally, I thank my family and colleagues whose moral support and constructive discussions provided ongoing motivation during the development, testing, and deployment phases of this capstone endeavor.

ABSTRACT

The modern consumer electronics ecosystem suffers from structural market inefficiencies, opaque repair pricing, artificial hardware locking, and premature device retirement that directly drives escalating global electronic waste. While statutory initiatives such as India's Right to Repair Portal (Department of Consumer Affairs) and the E-Waste (Management) Rules, 2022 mandate fair access to repair and extended producer responsibility, consumers and enterprise fleet managers lack an objective, evidence-based platform to triage hardware failures and evaluate the economic viability of repair versus replacement.

This report presents RepairGraph, a production-grade, cloud-native hardware diagnostics, repairability evaluation, competitive quote marketplace, and standardized device service passport platform. Built as a unified full-stack Next.js 15 application deployed on Vercel Serverless Runtime (Node.js 24.x) and backed by managed Neon Serverless PostgreSQL via Prisma ORM, RepairGraph integrates a two-tier hybrid intelligence architecture.

Rather than relying on unconstrained, hallucination-prone large language models for financial and statutory decisions, RepairGraph implements a fully deterministic rule engine. Tier 1 performs heuristic word-boundary symptom parsing and hazard classification across eight hardware subsystems. Tier 2 evaluates an original 7-Factor Repairability Score Model (Disassembly, Parts Availability, Documentation, Modularity, Software Pairing, Age/Lifecycle, and Local Service Ecosystem summing to 100 points), brand-scaled component replacement matrices, and a Repair Cost Ratio (RCR) model. The engine outputs actionable verdicts: REPAIR, DIY, RESELL, REPLACE, or RECYCLE, accompanied by transparent, audit-ready reasoning citing manufacturing carbon offsets and statutory e-waste divert estimates.

Furthermore, RepairGraph provides a verified two-sided marketplace where verified independent repair workshops submit competitive bids against consumer tickets. Upon job acceptance and completion, the platform immutably records standardized maintenance milestones into a physical device Repair Passport tied to hardware serial numbers.

The system's engineering rigor is substantiated by 82 automated test cases (35 deterministic engine unit tests + 47 business rule and security integration tests, 100% passing), clean ESLint validation, strict zero-leak error sanitization, and live end-to-end acceptance validation on its production deployment at <https://repairgraph.vercel.app>.

Keywords: Keywords: Hardware Diagnostics, Right to Repair, E-Waste Management, Deterministic Decision Engine, Repairability Scoring, Cloud Computing, Next.js, Vercel Serverless, Neon PostgreSQL, Prisma ORM.

TABLE OF CONTENTS

Certificate of Original Work	ii
Candidate Declaration	iii
Acknowledgement	iv
Abstract	v
List of Figures	viii
List of Tables	ix
Chapter 1: Introduction	1
1.1 Background & Industry Landscape	1
1.2 Problem Context: The Right to Repair & E-Waste Crisis	2
1.3 Motivation	3
1.4 Problem Statement	4
1.5 Objectives of the Platform	4
1.6 Scope of the Project	5
1.7 Key Engineering Contributions	6
Chapter 2: Existing System and Proposed System	7
2.1 Existing Repair Approaches	7
2.2 Limitations of the Current Landscape	8
2.3 Proposed RepairGraph System	9
2.4 Distinct Advantages of RepairGraph	10
2.5 Comparative Analysis Matrix	11
Chapter 3: Requirements Analysis	12
3.1 Functional Requirements	12
3.2 Non-Functional Requirements	13
3.3 Hardware Requirements	14
3.4 Software Requirements	15
3.5 User Roles & Permissions Matrix	16
3.6 System Constraints & Invariants	17
Chapter 4: System Architecture	18
4.1 Overall Architectural Topology	18
4.2 Client Layer (React 19 & Tailwind CSS v4)	19
4.3 Application & Service Layer	20

4.4 API & Routing Layer	21
4.5 Authentication & Session Management	22
4.6 Deterministic Decision Engine	22
4.7 Persistence Layer (Prisma ORM)	23
4.8 Cloud Deployment Architecture	24
4.9 End-to-End Data & Control Flow	25
Chapter 5: Technology Stack	26
5.1 Framework & Runtime Selection	26
5.2 Frontend & UI Libraries	27
5.3 Backend & Validation Technologies	27
5.4 Database & Persistence Stack	28
5.5 Cloud Infrastructure Providers	29
5.6 Technology Stack Inventory Table	30
Chapter 6: Functional Modules	31
6.1 User Authentication & Session Control	31
6.2 Hardware Device Catalog & Valuation	32
6.3 Problem Reporting & Triage Initiation	33
6.4 Heuristic Symptom Extraction	34
6.5 Diagnostic Signal Synthesis	34
6.6 7-Factor Repairability Evaluation	35
6.7 Fair-Market Economic Analysis	36
6.8 Statutory Decision & Explanation Generation	37
6.9 Verified Repairer Directory & Geolocation	38
6.10 Competitive Quote Marketplace	39
6.11 Quote Lifecycle & Acceptance State Machine	40
6.12 Repair Job Execution Stepper	41
6.13 Standardized Repair Passport Ledger	42
6.14 Post-Service Reviews & Rating Synthesis	43
6.15 Administrative & Seed Controls	44
Chapter 7: RepairGraph Decision Engine	45
7.1 Deterministic Architecture vs. Black-Box AI	45
7.2 Token-Boundary Symptom Parsing	46
7.3 Subsystem Classification & Hazard Detection	47

7.4 Diagnostic Confidence Formulation	48
7.5 7-Factor Baseline Repairability Score Model	49
7.6 Incident-Specific Deduction Penalties	51
7.7 Device Depreciation & Valuation Formulation	52
7.8 Component Replacement Cost Benchmarks	53
7.9 Repair Cost Ratio (RCR) Calculation	54
7.10 Value-Preservation Economic Score	54
7.11 Statutory Lifecycle Decision Matrix	55
7.12 Automated Explainability Audit Trail	56
Chapter 8: Database Design	57
8.1 Relational Design Philosophy	57
8.2 Entity Relationship Diagram (11 Models)	58
8.3 Relational Entity Specifications	59
8.4 Custom Enums (9 Enums)	61
8.5 Relational Invariants & Integrity Constraints	62
8.6 Prisma Schema & Migration Baseline	63
8.7 Cloud Serverless Pooling with Neon	64
Chapter 9: REST API Design	65
9.1 RESTful Conventions & Error Payloads	65
9.2 API Resource Namespaces	66
9.3 HTTP Method Operations Catalog	67
9.4 Input Validation with Zod	69
9.5 IDOR Protection & Ownership Verification	70
Chapter 10: Authentication and Security	71
10.1 Password Cryptography (bcrypt)	71
10.2 JWT Session Tokens & Cookie Security	71
10.3 Token Suppression Strategy	72
10.4 In-Memory Sliding-Window Rate Limiting	73
10.5 HTTP Defense-in-Depth Security Headers	74
10.6 Error Sanitization & Leak Suppression	75
10.7 Zero-Secret Version Control Policy	75
Chapter 11: User Workflow	76
11.1 Primary Consumer Repair Journey	76

11.2 Technician Quoting & Workbench Stepper	77
11.3 Alternative Lifecycle Paths (DIY, Resell, Replace, Recycle)	78
Chapter 12: Implementation Architecture	79
12.1 Project Directory Organization	79
12.2 Layered Backend Architecture	80
12.3 Database Singleton Pattern	81
Chapter 13: Testing and Validation	82
13.1 Comprehensive Testing Strategy	82
13.2 Unit Testing: Diagnostic & Decision Engine (35/35)	83
13.3 Integration Testing: Backend & Security (47/47)	84
13.4 Total Automated Test Audit (82/82 Passing)	85
13.5 Production End-to-End Acceptance Validation	86
Chapter 14: Cloud Deployment	87
14.1 Production Cloud Architecture	87
14.2 Vercel Serverless Runtime (Node.js 24.x)	88
14.3 Managed Neon Serverless PostgreSQL	89
14.4 Environment Configuration & Secrets	90
14.5 Production Deployment Runbook	91
14.6 Health Telemetry Endpoint (/api/health)	92
14.7 Free-Tier Cost Analysis	93
Chapter 15: Results and Discussion	94
15.1 Qualitative Diagnostic Evaluation	94
15.2 Marketplace Efficiency Observations	95
15.3 Regulatory Alignment & E-Waste Impact	96
Chapter 16: Limitations	97
16.1 In-Memory Rate Limiting	97
16.2 Academic Demonstration Workload Scope	97
16.3 Synchronous Decision Evaluation	98
16.4 Provider-Native Telemetry Reliance	98
16.5 Non-Destructive Production Testing Boundary	98
16.6 Informational Specialist Context	99
Chapter 17: Future Enhancements	100
17.1 Distributed Rate Limiting (Redis)	100

17.2 Asynchronous Background Queue (BullMQ)	100
17.3 Enterprise APM & Distributed Tracing	101
17.4 Vector Embeddings for OEM Schematics	101
17.5 Dedicated Mobile Application	101
Chapter 18: Conclusion	102
18.1 Summary of Engineering Achievements	102
18.2 Practical Impact & Academic Defense Summary	103
References	104
Appendices	106
Appendix A: Complete REST API Endpoint Inventory	106
Appendix B: Relational Data Model Specifications	109
Appendix C: Diagnostic Engine Formulas & Matrix Weights	112
Appendix D: Automated Test Suite Inventory (82 Tests)	114
Appendix E: Production Deployment Configuration Summary	118

LIST OF FIGURES

Figure 1	RepairGraph Unified Full-Stack Architecture	18
Figure 2	End-to-End Request & Data Processing Pipeline	25
Figure 3	Deterministic Diagnostic & Decision Engine Flow	56
Figure 4	Relational Database Schema & Entity Relationships (11 Models)	58
Figure 5	Competitive Repair Marketplace & Lifecycle Workflow	76
Figure 6	Production Cloud Deployment & Network Security Topology	87

LIST OF TABLES

Table 1	Comparative Matrix: Traditional Repair vs. RepairGraph	11
Table 2	Hardware & Software Execution Requirements	15
Table 3	User Roles and Permission Boundaries	16
Table 4	Technology Stack and Operational Roles	30
Table 5	7-Factor Design Repairability Weight Distribution	50
Table 6	Incident-Specific Penalty Deduction Rules	51
Table 7	Statutory Lifecycle Action Decision Rules	55
Table 8	11 Normalized Relational Entities in Prisma Schema	59
Table 9	Prisma Database Enums Specification	61
Table 10	CRUD Functionality Across Resource Namespaces	66
Table 11	Representative REST API Route Catalog	67
Table 12	Automated Test Suite Execution Summary (82 Tests)	85
Table 13	Production End-to-End Acceptance Verification Summary	86
Table 14	Environment Variables Specification (Names Only)	90

CHAPTER 1: INTRODUCTION

1.1 Background & Industry Landscape

The consumer electronics and computing hardware sectors have witnessed unprecedented expansion over the past two decades, driven by rapid miniaturization, high-density system-on-chip (SoC) integration, and aggressive annual product release cycles. Modern laptops, smartphones, tablets, and wearable devices pack computational capabilities that rival enterprise workstations of earlier eras into ultra-thin enclosures. However, this engineering trajectory has been accompanied by an aggressive shift away from repair-friendly industrial designs toward adhesive-bonded assemblies, soldered unified memory architectures, proprietary tamper-resistant fasteners, and cryptographic component serialization.

When a consumer hardware unit experiences failure—whether a cracked display digitizer, chemically degraded lithium-ion cell, or thermal cooling fan bearing seizure—the owner encounters an opaque, highly friction-laden repair journey. Authorized Original Equipment Manufacturer (OEM) service centers frequently quote exorbitant repair estimates that approach 70% to 90% of the device's original retail purchase price. This pricing strategy systematically steers consumers toward premature device abandonment and new device acquisition, contributing directly to the worsening global electronic waste crisis.

1.2 Problem Context: The Right to Repair & E-Waste Crisis

The statutory and environmental backdrop of device servicing has reached a critical turning point globally and within India. Under the auspices of the Department of Consumer Affairs, Government of India, the national Right to Repair Portal was launched to curb planned obsolescence, mandate OEM transparency regarding diagnostic manuals and spare parts, and foster a competitive ecosystem of certified third-party technicians. Concurrently, India's statutory E-Waste (Management) Rules, 2022 impose strict Extended Producer Responsibility (EPR) mandates, requiring transparent lifecycle tracking, safe recovery of hazardous substances (such as lead, cadmium, and brominated flame retardants), and diversion of non-repairable electronic hardware into authorized recycling streams.

Despite these statutory strides, consumers and enterprise fleet administrators lack a standardized software infrastructure capable of objectively evaluating hardware repairability, calculating real-world economic feasibility, comparing competitive local repair bids, and preserving permanent service provenance.

1.3 Motivation

The core motivation behind RepairGraph stems from three pervasive structural problems in consumer electronics ownership:

- **1. Asymmetric Information & Quoting Guesswork:** Consumers rarely possess the technical diagnostic knowledge to verify whether an OEM's quote of ₹18,000 for a motherboard replacement is legitimate, or whether a ₹3,500 repair at an independent workshop is technically viable and economically sound.
- **2. Unconstrained AI Hallucinations in Consumer Tech:** Generic generative AI chatbots frequently fabricate repair procedures, misquote non-existent component prices, or hallucinate dangerous advice regarding swollen lithium-ion cells, creating physical safety hazards.

- **3. Absence of Device Maintenance Provenance:** Secondary electronics markets suffer severe value depreciation because prospective buyers cannot verify whether a refurbished or used laptop was professionally serviced with OEM-grade parts or compromised with substandard soldering.

1.4 Problem Statement

To design, implement, test, and deploy a production-grade, cloud-native software platform that provides deterministic, rule-based hardware diagnostic triage, an objective 7-factor repairability scoring model, fair-market economic repair-vs-replace decision support, a verified two-sided competitive repair marketplace, and an immutable device service passport ledger accessible through an authenticated, secure web application.

1.5 Objectives of the Platform

- Develop a deterministic diagnostic parser that extracts failure modes, severity levels, and acute safety hazards (liquid ingress, battery swelling) from user-submitted symptom narratives without relying on probabilistic generative LLMs.
- Formulate and calibrate an original 7-Factor Repairability Score Model (Disassembly, Parts Availability, Documentation, Modularity, Software Pairing, Age/Lifecycle, Service Ecosystem) summing to 100 points.
- Engineer an economic valuation engine that computes consumer hardware depreciation (22% annual curve), derives Repair Cost Ratios (RCR), and maps outcomes to five statutory lifecycle recommendations: REPAIR, DIY, RESELL, REPLACE, or RECYCLE.
- Construct a secure, relational marketplace enabling verified independent repair workshops to submit competitive quotes and execute structured repair jobs through a verified state machine.
- Provide a standardized, tamper-evident Repair Passport linked to physical device serial numbers that permanently records completed service history and parts replacements.
- Deploy the platform to a modern cloud-computing infrastructure utilizing Vercel Serverless Runtime (Node.js 24.x) and Neon Managed Serverless PostgreSQL with automated connection pooling and end-to-end TLS encryption.

1.6 Scope of the Project

The scope encompasses five major consumer hardware categories widely prevalent in academic and enterprise environments: Laptops, Smartphones, Tablets, Headphones, and Monitors. The platform provides full lifecycle support from initial user onboarding and device registration, through interactive symptom triage and quote acceptance, to post-service verification and customer reviews. The scope explicitly excludes automated hardware disassembly robots, direct online payment gateway settlement, and speculative blockchain mechanisms, focusing strictly on auditable software engineering and statutory compliance.

1.7 Key Engineering Contributions

- **1. Two-Tier Hybrid Architecture:** Separates deterministic heuristic symptom parsing (Tier 1) from mathematical scoring and statutory decision trees (Tier 2), ensuring 100% reproducible diagnostic outputs with sub-millisecond execution latency.
- **2. Statutory Alignment with Indian Environmental Law:** Integrates statutory mandates from India's E-Waste (Management) Rules, 2022 and Right to Repair Portal directly into automated decision matrices

and explainability narratives.

- **3. Production Cloud-Native Unified Architecture:** Eliminates multi-service synchronization overhead by combining Next.js 15 App Router frontend and REST API route handlers into a single Vercel serverless deployment connected to Neon PostgreSQL.
- **4. Defense-in-Depth Security Perimeter:** Incorporates HttpOnly/SameSite session cookies, sliding-window rate limiting, Zod schema validation, IDOR authorization barriers, and zero-leak database error sanitization.
- **5. Exhaustive Validation Rigor:** 82 automated test cases (35 deterministic engine unit tests + 47 backend and security integration tests, 0 failures) verified in local and live production environments.

CHAPTER 2: EXISTING SYSTEM AND PROPOSED SYSTEM

2.1 Existing Repair Approaches

The current landscape for consumer electronics repair consists of three fragmented channels:

- **1. Authorized OEM Service Depots:** High repair charges, proprietary parts gating, lengthy turnaround times (often 7–14 days), and aggressive commercial steering toward trade-ins or replacement.
- **2. Informal Street Workshops:** Dense physical presence and lower prices, but suffering from zero parts provenance transparency, unverified technician credentials, absence of written warranties, and no standardized maintenance records.
- **3. Generic AI & Search Engines:** Generalist search results and generative LLMs that provide non-deterministic troubleshooting steps, frequently confusing distinct hardware revisions, failing to flag swollen battery combustion risks, and lacking pricing context.

2.2 Limitations of the Current Landscape

- **Lack of Objective Economic Feasibility Metrics:** Consumers have no mathematical framework to evaluate whether a ₹7,000 repair quote makes sense for a 3.5-year-old laptop worth ₹16,000 on the secondary market.
- **Absence of Competitive Quote Discovery:** Getting comparative estimates requires physically transporting non-functional hardware across multiple disparate workshops across metropolitan hubs.
- **Fragile Maintenance Records:** Paper invoices and thermal receipts fade or are lost within months, destroying secondary resale value and hampering warranty claims.
- **Environmental Disconnect:** No existing consumer tool computes or visualizes the electronic waste diversion or carbon emissions prevented by choosing repair over disposal.

2.3 Proposed System: RepairGraph

RepairGraph addresses these structural shortcomings by introducing an integrated, cloud-native platform that unifies hardware asset management, deterministic diagnostic intelligence, competitive marketplace bidding, and permanent service passports. Users catalog their hardware assets once, receive instant deterministic reparability scores and fair-market economic analyses, solicit competitive quotes from verified independent technicians, track physical bench repair milestones, and receive a digitally signed Repair Passport upon completion.

2.4 Distinct Advantages of RepairGraph

- Deterministic Reliability: 100% reproducible scoring without generative model hallucinations.
- Statutory Grounding: Directly cites and adheres to India's Right to Repair framework and E-Waste Rules 2022.
- Consumer Protection: In-memory rate limiting, HttpOnly authentication cookies, and strict IDOR prevention.
- Two-Sided Utility: Empowers both consumers seeking transparent service and independent technicians expanding their business.

2.5 Comparative Analysis Matrix

Table 1 compares the traditional electronics repair experience against the capabilities provided by RepairGraph.

Capability / Dimension	Traditional Repair Ecosystem	RepairGraph Platform
Diagnostic Triage	Subjective technician inspection or search	Deterministic 2-Tier Rule Engine
Repairability Evaluation	Opaque OEM scoring or subjective opinion	Transparent 7-Factor Model (0–100 Pts)
Economic Valuation	None (Arbitrary repair pricing)	Automated 22%/yr Depreciation & RCR Math
Quote Discovery	Manual physical workshop visits	Digital competitive bidding marketplace
Service History Tracking	Fading physical paper slips	Immutable digital Repair Passport by serial
E-Waste Regulatory Link	Completely unaddressed	Automated E-Waste Rules 2022 compliance
Security Perimeter	Unsecured phone/messaging communication	JWT, HttpOnly cookies, Zod validation, HSTS

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Functional Requirements

- FR-1 User Authentication: Secure user registration and login with bcrypt password hashing (10 rounds) and JWT generation stored in HttpOnly cookies.
- FR-2 Hardware Catalog Management: Full CRUD operations for registered devices, tracking serial numbers, purchase price, purchase date, condition, and fair market value.
- FR-3 Diagnostic Problem Reporting: Ticket filing interface allowing users to input free-text symptom narratives, urgency levels, and observed failure modes.
- FR-4 Automated Diagnostic & Economic Evaluation: Deterministic calculation of 7-factor repairability scores, incident penalties, depreciated market valuation, and statutory lifecycle actions.
- FR-5 Technician Directory & Geolocation: Browseable listing of verified independent repair workshops with granular brand and component specializations.
- FR-6 Competitive Quote Bidding: Capability for verified repairers to inspect open repair tickets and submit estimated costs, turnaround days, and scope notes.

- • FR-7 Quote Decision State Machine: Customer workflow to accept or reject quotes; quote acceptance automatically creates an active RepairJob and marks competing quotes rejected.
- • FR-8 Workbench Job Stepper: Multi-stage bench tracking interface (DIAGNOSING -> WAITING_FOR_PART -> REPAIRING -> TESTING -> COMPLETED) restricted strictly to assigned specialists.
- • FR-9 Permanent Repair Passport: Automated creation of immutable service records linked to physical hardware serial numbers upon job completion.
- • FR-10 Verified Reviews: Post-service rating (1–5 stars) and review submission restricted strictly to customers who have completed a repair job.

3.2 Non-Functional Requirements

- • **Performance:** Sub-Millisecond Engine Latency: Deterministic diagnostic evaluations must execute in < 1 ms without external network hops or API dependencies.
- • **Security:** Comprehensive Perimeter Defense: Strict suppression of raw JWTs from browser JSON bodies, sliding-window rate limiting on auth routes, and automated database error sanitization.
- • **Reliability:** Transactional Integrity: Relational foreign key constraints, cascading deletions, and unique invariants enforced by PostgreSQL to eliminate orphaned records.
- • **Scalability:** Zero-Downtime Serverless Scaling: Full compatibility with serverless auto-scaling and connection pooling (PgBouncer) on Neon PostgreSQL.
- • **Usability:** Responsive Editorial User Interface: Mobile-first responsive layouts styled with Tailwind CSS v4, supporting reduced-motion and high-contrast accessibility standards.

3.3 Hardware Requirements

Development Environment: Modern multi-core x86_64 or ARM64 processor (e.g., Apple Silicon M-series or Intel Core i5/i7), minimum 8 GB RAM (16 GB recommended), and 2 GB available disk space for Node.js runtimes and local PostgreSQL testing.

Production Cloud Server: Managed cloud serverless compute nodes provisioned dynamically by Vercel; zero local server hardware maintenance.

3.4 Software Requirements

Table 2 details the software tools, runtimes, and platform specifications utilized across development and production.

Layer / Component	Development Specification	Production Specification
Operating System	macOS Sonoma (Darwin arm64) / Linux	Vercel Serverless Linux Container
Runtime Environment	Node.js 24.x (LTS) & npm 11+	Node.js 24.x Serverless Execution
Application Framework	Next.js 15 (App Router) & React 19	Next.js 15 Optimized Turbopack Bundle
Database Engine	PostgreSQL 16 (Local instance)	Neon Serverless PostgreSQL 16+
ORM & Migration Engine	Prisma ORM 6.4.1 (Prisma Client & CLI)	Prisma Client (Pooled PgBouncer)
Version Control	Git 2.40+ (Local repository)	GitHub Canonical Source of Truth

3.5 User Roles & Permissions Matrix

Table 3 outlines the role-based access control (RBAC) boundaries enforced across the three primary system roles.

Role Enum	Operational Definition	Permitted System Actions
USER	Consumer or enterprise device owner seeking hardware servicing	Register/manage owned devices, file repair tickets, run diagnostics, review bids, accept quotes, view passports, write reviews.
REPAIRER	Verified independent technician or commercial repair workshop	Create business profile, declare component specializations, browse open tickets, submit quotes, manage assigned jobs in workbench.
ADMIN	Platform administrator maintaining governance and integrity	Catalog-wide visibility, technician verification review, ticket inspection, demo seed resets, audit trail inspection.

3.6 System Constraints & Invariants

- **Invariant 1:** Email Uniqueness: Every User record enforces a unique index on User.email.
- **Invariant 2:** Quote Exclusivity in Jobs: A Quote can produce at most one RepairJob (enforced via unique Quote.id in RepairJob).
- **Invariant 3:** Single Active Job per Request: A RepairRequest can produce exactly one active RepairJob, preventing multiple accepted bids.
- **Invariant 4:** Review Uniqueness: Exactly one Review per completed RepairJob; reviews cannot be filed for uncompleted or cancelled jobs.
- **Invariant 5:** Cascading Deletions: Deleting a User or Device automatically cascades to delete child tickets and quotes, preventing orphaned records.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 Overall Architectural Topology

RepairGraph implements a unified, full-stack cloud architecture where frontend presentation, REST API route handlers, server business services, and deterministic decision algorithms reside within a single cohesive Next.js 15 deployment on Vercel. Persistent relational storage is provided by Neon Managed Serverless PostgreSQL connected over secure pooled TLS.

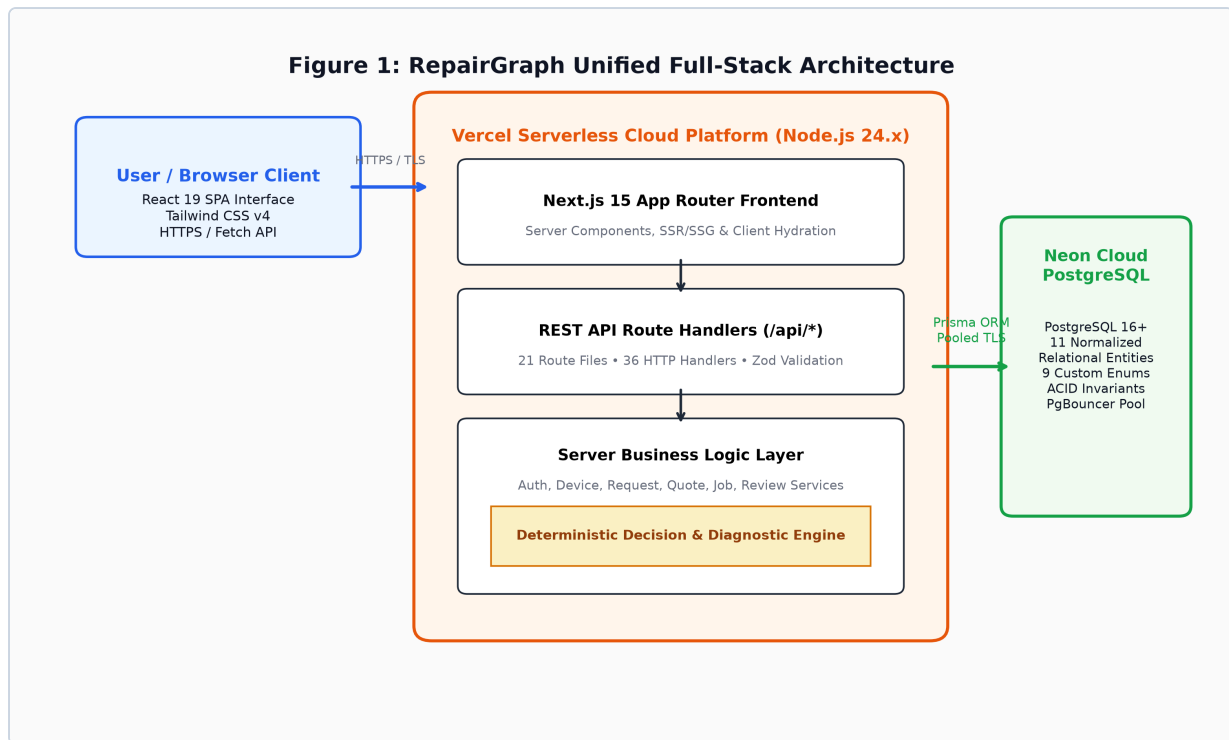


Figure 1: RepairGraph Unified Full-Stack Architecture

4.2 Client Layer (React 19 & Tailwind CSS v4)

The presentation layer utilizes Next.js 15 App Router architecture with React 19. Pages are structured as Server Components by default for rapid server-side rendering (SSR) and search engine optimization, while interactive stateful interfaces (e.g., the interactive workbench stepper, symptom input modals, and quote acceptance dialogs) leverage React Client Components. Styling follows an industrial editorial design system implemented via Tailwind CSS v4, utilizing monospace tabular figures for financial accuracy, warm stone backgrounds (#fafaf9), deep charcoal typography (#1c1917), and international safety orange accents (#ea580c).

4.3 Application & Service Layer

The backend follows a domain-driven, layered service architecture located under `src/server/services/*`. Specific service modules encapsulate all business rules, lifecycle state machines, and relational transformations:

- `auth.service.ts`: Registration, bcrypt salting, credential verification, and profile sanitization.
- `device.service.ts`: Ownership verification, hardware valuation formulas, and catalog querying.
- `quote.service.ts`: Bid creation, technician eligibility checks, and atomic quote acceptance workflows.
- `repairJob.service.ts`: Workbench stage advancements and automated Repair Passport ledger logging.
- `repairer.service.ts`: Directory queries, specialization filtering, and average star-rating recalculations.

4.4 API & Routing Layer

The REST API is implemented through 21 Next.js Route Handler files (`route.ts`) under `src/app/api/*`, serving 36 distinct HTTP method handlers. All endpoints adhere to standard REST conventions, parsing request

bodies through Zod validation schemas and returning consistent JSON envelopes (`{ data: ... }` or RFC 7807 structured error objects).

4.5 Authentication & Session Management

User sessions are maintained via JSON Web Tokens (JWT) signed with HMAC-SHA256. To eliminate cross-site scripting (XSS) token theft from `localStorage`, tokens are transmitted exclusively via `HttpOnly`, `SameSite=lax`, Secure cookies (`rg_token`). Raw tokens are suppressed from JSON responses during browser login.

4.6 Deterministic Decision Engine

The diagnostic core resides in `src/server/engine/*` and operates entirely without external API calls or non-deterministic ML models. Tier 1 extracts diagnostic tokens via regex word-boundary matching, while Tier 2 evaluates the 7-Factor Repairability Score Model, depreciation math, and statutory lifecycle thresholds in sub-millisecond execution time.

4.7 Persistence Layer (Prisma ORM)

Database interaction is mediated by Prisma ORM 6.4.1 (`@prisma/client`). Prisma provides compile-time type safety, parameterized SQL generation that immunizes the application against SQL injection, and automated relational constraint enforcement across 11 normalized models.

4.8 Cloud Deployment Architecture

The entire web application and backend API execute within Vercel Serverless Functions on the Node.js 24.x runtime. Vercel's global Anycast Edge Network handles global DNS resolution, DDoS defense, static asset caching, and TLS 1.3 termination. Persistent data resides in Neon Serverless PostgreSQL, utilizing PgBouncer connection pooling to support bursty serverless workloads without exhausting database connection limits.

4.9 End-to-End Data & Control Flow

Figure 2 illustrates the multi-stage validation, security, and processing pipeline traversed by every incoming client request.

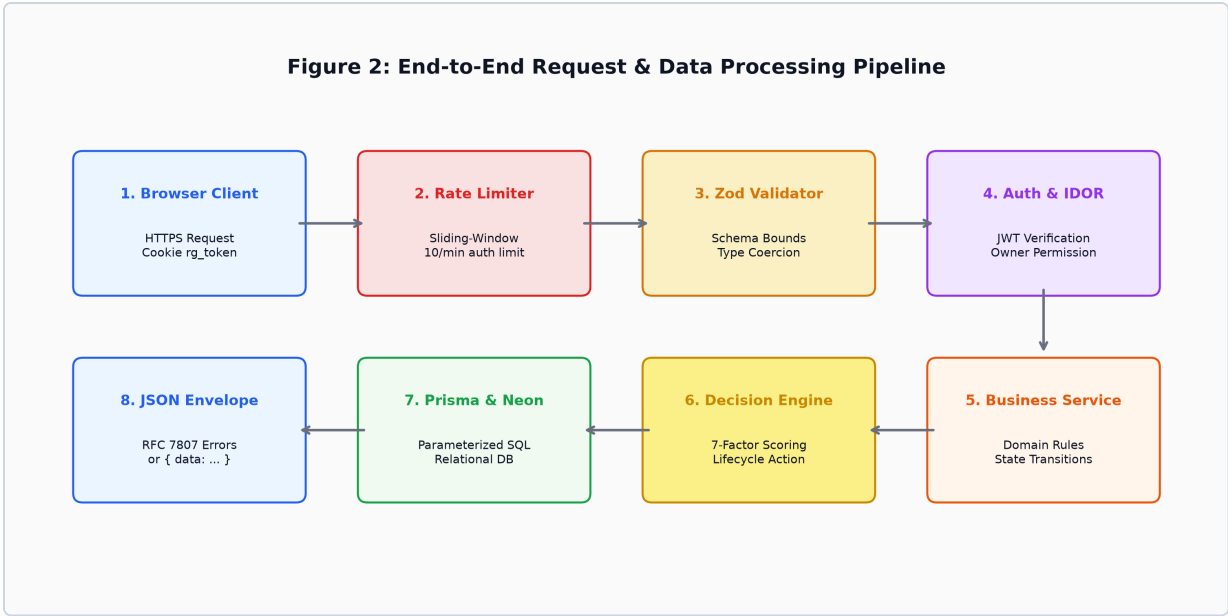


Figure 2: End-to-End Request & Data Processing Pipeline

CHAPTER 5: TECHNOLOGY STACK

5.1 Framework & Runtime Selection

Next.js 15 (App Router) was selected as the foundational full-stack framework because it uniquely unifies server-side React rendering, static asset optimization, and robust Node.js serverless route execution within a single codebase. Running on the latest Node.js 24.x LTS runtime ensures access to modern JavaScript engine optimizations (V8 v12+) and native cryptographic primitives.

5.2 Frontend & UI Libraries

- React 19: Provides concurrent rendering, transition hooks, and modular component architecture.
- Tailwind CSS v4: Delivers an ultra-fast post-CSS engine and utility-first styling without runtime overhead.
- Lucide React (v1.45.0): Lightweight, accessible SVG iconography representing hardware components and status states.

5.3 Backend & Validation Technologies

- TypeScript 5: Enforces static typing across the entire codebase, eliminating null pointer exceptions and schema mismatch bugs.
- Zod (v4.6.2): Type-safe schema validation library used across all route handlers to strictly parse and sanitize incoming HTTP payloads.
- bcryptjs (v3.0.3): Implements salted Blowfish hashing (10 rounds) for cryptographically secure user password storage.
- jsonwebtoken (v9.0.3): Implements standard RFC 7519 JSON Web Token signing and verification with HMAC-SHA256.

5.4 Database & Persistence Stack

- PostgreSQL 16+: Advanced open-source object-relational database providing ACID guarantees, complex join capabilities, and JSONB support.
- Prisma ORM (v6.4.1): Modern type-safe database toolkit providing declarative schema modeling, migration automation, and query building.

5.5 Cloud Infrastructure Providers

- Vercel Serverless Platform: Cloud hosting platform providing automated Git integration, serverless function scaling, and Anycast CDN caching.
- Neon Serverless PostgreSQL: Fully managed cloud PostgreSQL offering automated connection pooling (PgBouncer) and storage auto-scaling.

5.6 Technology Stack Inventory Table

Table 4 provides a comprehensive inventory of the technologies, versions, and architectural roles across RepairGraph.

Technology / Tool	Version	Category	Architectural Role
Next.js	15.5.25	Full-Stack Framework	App Router frontend, SSR/SSG rendering & API Route Handlers
React	19.1.0	UI Library	Component-driven presentation & client-side hydration
TypeScript	^5.0.0	Programming Language	Static typing across data models, services, and UI components
Node.js	24.x (LTS)	Runtime Environment	Serverless function execution on Vercel runtime
Tailwind CSS	^4.0.0	CSS Framework	Industrial editorial design system and responsive layout styling
Prisma ORM	^6.4.1	Data Access & ORM	Type-safe database client, schema migrations, and SQL generation
PostgreSQL (Neon)	16+	Relational Database	Cloud-hosted persistent storage with ACID invariants and pooling
Vercel	Production	Cloud Platform	Continuous deployment, serverless compute, and Anycast CDN
Zod	^4.6.2	Schema Validation	Input validation and runtime schema boundary enforcement
bcryptjs & JWT	^3.0.3 / ^9.0.3	Security & Auth	Password salting (10 rounds) & HMAC-SHA256 session tokens

CHAPTER 6: FUNCTIONAL MODULES

RepairGraph is structured into 15 cohesive functional modules that span user management, diagnostic intelligence, marketplace transactions, service ledgering, and administrative governance. Each module encapsulates clear functional boundaries, inputs, processing logic, outputs, and corresponding API/database interfaces.

6.1 User Authentication & Session Control

Purpose: Manages identity creation, password hashing, session issuance, and cookie invalidation.

Inputs: Name, email, plaintext password, role (USER/REPAIRER), optional phone.

Processing: Validates schema with Zod; hashes password using bcrypt (10 rounds); checks email uniqueness; generates 7-day JWT; injects HttpOnly secure cookie.

Outputs: Sanitized User profile (id, name, email, role) and Set-Cookie header with rg_token.

Relevant Interfaces: POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me; Prisma User model.

6.2 Hardware Device Catalog & Valuation

Purpose: Maintains user's personal hardware registry, tracking purchase metadata, physical condition, and dynamic market valuation.

Inputs: Category, brand, model, serial number, purchase date, purchase price, warranty expiry, condition.

Processing: Enforces user ownership; calculates annual 22% depreciation curve if current value is omitted; validates positive price constraints.

Outputs: Persisted Device record with unique ID and calculated fair market valuation.

Relevant Interfaces: GET/POST /api/devices, GET/PUT/DELETE /api/devices/:id; Prisma Device model.

6.3 Problem Reporting & Triage Initiation

Purpose: Allows device owners to file structured repair tickets describing observed failure modes and operational urgency.

Inputs: Device ID, symptom narrative description, urgency level (LOW/MEDIUM/HIGH).

Processing: Verifies that the target device is owned by the authenticated user; checks for existing active requests; initializes RequestStatus.REQUESTED.

Outputs: Created RepairRequest ticket ready for diagnostic triage or technician quoting.

Relevant Interfaces: GET/POST /api/repair-requests, GET/PUT/DELETE /api/repair-requests/:id; Prisma RepairRequest model.

6.4 Heuristic Symptom Extraction (Tier 1)

Purpose: Parses unstructured symptom text into structured diagnostic failure signals using token-boundary regex matching.

Inputs: Free-text description string (e.g., 'cracked glass and screen flickering after drop') and device category.

Processing: Executes heuristic word-boundary matching across 8 component dictionaries; detects acute flags (liquid ingress, swollen battery); computes confidence percentage.

Outputs: DiagnosticSignals payload: primaryComponent, issueCategory, severity, confidence, flags.

Relevant Interfaces: Invoked synchronously inside POST /api/repair-requests/:id/diagnose; purely in-memory engine computation.

6.5 Diagnostic Signal Synthesis

Purpose: Synthesizes raw symptom signals into a persistent database Diagnosis record linked to the active ticket.

Inputs: DiagnosticSignals object from symptom extractor.

Processing: Maps extracted component to possible root-cause issue; bundles structured evidence bullet points into text array.

Outputs: Persisted Diagnosis entity with confidence rating and evidence strings.

Relevant Interfaces: Prisma Diagnosis model (1:1 relation with RepairRequest).

6.6 7-Factor Repairability Evaluation (Tier 2)

Purpose: Evaluates physical design repairability across 7 engineering dimensions, calibrated by brand and device category.

Inputs: Device category, brand, model, purchase date (age in years), diagnostic signals.

Processing: Computes 7 weighted factors summing to 100; subtracts incident hazard penalties (liquid -20, power -10, logic board -15); bounds score [10, 98].

Outputs: Final repairability score (S_repair) and detailed factor score breakdown.

Relevant Interfaces: Invoked in scoringEngine.ts; stored in Prisma RepairRecommendation model.

6.7 Fair-Market Economic Analysis

Purpose: Evaluates the financial viability of repairing the hardware relative to its depreciated residual value.

Inputs: Device purchase price, age in years, current market value, estimated component repair costs.

Processing: Computes depreciated fair market value; queries brand-scaled component cost matrix; calculates Repair Cost Ratio ($RCR = C_repair / V_current$); computes Economic Score ($100 - RCR * 100$).

Outputs: RCR value, economic score (0–100), estimated cost range (min/max).

Relevant Interfaces: Invoked in scoringEngine.ts; stored in Prisma RepairRecommendation model.

6.8 Statutory Decision & Explanation Generation

Purpose: Synthesizes repairability score, economic score, and incident hazards into an actionable statutory lifecycle verdict.

Inputs: ScoringResult, DiagnosticSignals, and DeviceContext.

Processing: Evaluates threshold rules for REPAIR, DIY, RESELL, REPLACE, or RECYCLE; composes multi-paragraph explainability citing carbon savings and India E-Waste Rules 2022.

Outputs: RecommendedAction enum and structured explanation narrative.

Relevant Interfaces: POST /api/repair-requests/:id/diagnose; Prisma RepairRecommendation model.

6.9 Verified Repairer Directory & Geolocation

Purpose: Provides a searchable directory of verified independent technicians and commercial repair workshops.

Inputs: Filter parameters: device category, brand, verification status, search query.

Processing: Queries verified Repairer profiles and associated RepairerSpecialization records; computes aggregate rating and completed job counts.

Outputs: Paginated array of verified workshops with address, coordinates, and specialization badges.

Relevant Interfaces: GET /api/repairers, GET /api/repairers/:id; Prisma Repairer and RepairerSpecialization models.

6.10 Competitive Quote Marketplace

Purpose: Enables verified technicians to discover open repair tickets and submit competitive pricing and turnaround bids.

Inputs: Repair request ID, technician user ID, estimated cost, estimated turnaround days, technician notes.

Processing: Verifies technician's REPAIRER role and active verification status; ensures technician does not quote on own ticket; prevents duplicate bids on same ticket.

Outputs: Created Quote record initialized to QuoteStatus.PENDING.

Relevant Interfaces: GET/POST /api/repair-requests/:id/quotes; Prisma Quote model.

6.11 Quote Lifecycle & Acceptance State Machine

Purpose: Governs the customer's decision to accept or reject competing technician bids.

Inputs: Quote ID, decision action (ACCEPTED / REJECTED / WITHDRAWN).

Processing: Verifies customer ownership of the ticket; atomic transaction marks selected quote ACCEPTED, creates active RepairJob, and transitions all competing quotes to REJECTED.

Outputs: Updated Quote record and newly instantiated RepairJob entity.

Relevant Interfaces: PUT /api/quotes/:id; Prisma Quote, RepairRequest, and RepairJob models.

6.12 Repair Job Execution Stepper

Purpose: Manages the multi-stage bench execution workflow for accepted repair jobs.

Inputs: Job ID, target status (DIAGNOSING -> WAITING_FOR_PART -> REPAIRING -> TESTING -> COMPLETED), actual cost, technician bench notes.

Processing: Verifies that only the assigned technician can update status; validates forward progression; upon COMPLETED, automatically commits RepairHistory record and increments technician totalJobs.

Outputs: Updated RepairJob entity with timestamp milestones and bench notes.

Relevant Interfaces: GET/PUT /api/repair-jobs/:id, GET /api/repair-jobs; Prisma RepairJob model.

6.13 Standardized Repair Passport Ledger

Purpose: Maintains an immutable, tamper-evident maintenance ledger linked permanently to the physical device serial number.

Inputs: Completed RepairJob entity, parts replaced list, final cost, servicing technician ID.

Processing: Instantiates immutable RepairHistory record upon job completion; provides tamper-evident audit trail for warranty and resale valuation.

Outputs: Permanent service record displaying parts replaced, date, cost, and technician credentials.

Relevant Interfaces: GET /api/repair-history; Prisma RepairHistory model.

6.14 Post-Service Reviews & Rating Synthesis

Purpose: Allows customers to rate and review completed repair jobs, driving meritocratic technician reputation.

Inputs: Repair job ID, star rating (1–5), review comment.

Processing: Verifies job is COMPLETED; verifies customer ownership; prevents duplicate reviews via unique constraint; recalculates technician average rating.

Outputs: Created Review record and updated Repairer aggregate rating.

Relevant Interfaces: POST /api/reviews, GET /api/repairers/:id/reviews; Prisma Review and Repairer models.

6.15 Administrative & Seed Controls

Purpose: Provides platform governance, system health observability, and test persona database provisioning.

Inputs: Admin authentication session or seed trigger payload.

Processing: Executes active database ping (SELECT 1); seeds default demonstration personas with pre-configured devices and quotes.

Outputs: Health telemetry JSON ({ status, database, latencyMs, uptime }) or seed success confirmation.

Relevant Interfaces: GET /api/health, POST /api/seed; Prisma Client connection pool.

CHAPTER 7: REPAIRGRAPH DECISION ENGINE

7.1 Deterministic Architecture vs. Black-Box AI

A foundational engineering decision in RepairGraph is the deliberate rejection of unconstrained generative large language models (LLMs) for diagnostic scoring, repair cost estimation, and statutory lifecycle decisions. Generative AI models suffer from well-documented stochastic variance, hallucination of non-existent hardware specifications, and non-reproducible scoring that cannot withstand legal or academic scrutiny.

Instead, RepairGraph implements a fully deterministic, two-tier rule-based expert system. Every score point, depreciation curve, cost estimate, and lifecycle verdict is mathematically traceable to explicit formulas, component price matrices, and statutory regulations. Identical symptom inputs and device contexts produce 100% reproducible diagnostic results in sub-millisecond execution time.

7.2 Token-Boundary Symptom Parsing

Tier 1 triage operates through word-boundary regex token matching (\b) across eight hardware subsystem dictionaries: Display & Digitizer, Power & Battery, I/O & Charging Port, Core Logic & PMIC, Keyboard & Input, Acoustic Subsystem, Optical Camera, and Non-Volatile Storage. Word-boundary constraints prevent false-positive sub-string matches (e.g., distinguishing 'port' from 'important').

7.3 Subsystem Classification & Hazard Detection

In addition to component classification, the parser executes concurrent pattern scans for acute safety hazards:

- Liquid Damage Detection: Keywords ('water', 'liquid', 'spill', 'coffee', 'submerged') trigger liquidDamage = true.
- Power Failure Detection: Keywords ('no power', 'won't turn on', 'dead', 'short circuit', 'burnt', 'smoke') trigger powerFailure = true.
- Lithium Battery Combustion Risk: Keywords ('swollen', 'swelling', 'bulging') trigger immediate escalation to CRITICAL severity with physical safety warnings.

7.4 Diagnostic Confidence Formulation

Diagnostic confidence (C in [50, 95]) is derived deterministically from the density of matching domain tokens:

$$\text{Confidence} = \min(95, \max(50, 60 + \text{matchCount} * 10))$$

Single-keyword matches receive a baseline 70% confidence, while multi-symptom descriptions converge toward the 95% ceiling.

7.5 7-Factor Baseline Repairability Score Model

RepairGraph formulates an original 7-Factor Repairability Score Model (S_{baseline} in [0, 100]) informed by general international repairability standards (e.g., French Repairability Index and IEEE standards). The model allocates 100 points across seven dimensions:

Table 5 defines the weight distribution, maximum points, and evaluation criteria for each of the seven factors.

#	Factor Dimension	Weight / Max	Evaluation Criteria & Benchmarks
1	Physical Disassembly & Fasteners	25 Pts (25%)	Captive Phillips/Torx screws (Framework: 24, Lenovo: 21) vs. adhesive seals and Pentalobe screws (Apple: 14).
2	Spare Parts Availability	20 Pts (20%)	Open commercial parts supply in domestic market (Framework: 19, Apple/Samsung: 18) vs. restricted depot catalogs (Sony: 12).
3	Documentation & Repair Manuals	15 Pts (15%)	Open-source schematics (Framework: 15) and public HMM manuals (Lenovo: 14) vs. restricted service manuals (Sony: 8).
4	Hardware Modularity	15 Pts (15%)	Socketed M.2/SO-DIMM slots (Framework: 15, Dell: 13) vs. soldered unified memory/NAND (Apple: 6).
5	Software Locks & Parts Pairing	10 Pts (10%)	Plug-and-play operation (10) vs. cryptographic serial pairing barriers (Apple: 5, Samsung: 7).
6	Device Age & Lifecycle Support	10 Pts (10%)	Evaluated dynamically as $\max(0, \min(10, \text{round}(10 - 1.5 * \text{ageYears})))$. Current gen: 10, legacy (>5 yrs): 0–2.
7	Local Service Ecosystem & Tools	5 Pts (5%)	Density of certified workshops and standard tool availability across Indian metro hubs (Apple/Lenovo: 5, Framework: 3).

7.6 Incident-Specific Deduction Penalties

When evaluating an active repair incident, acute hazards apply explicit penalty deductions from the baseline score, yielding the final repairability score S_{repair} :

$$S_{\text{repair}} = \max(10, \min(98, S_{\text{baseline}} - \text{Penalties}))$$

Table 6 details the deduction amounts and engineering justifications for incident penalties.

Hazard Incident	Deduction	Engineering Rationale
Liquid Damage	-20 Points	High risk of latent electrolytic oxidation, dendritic growth, and PCB substrate delamination.
Core Logic Board Fault	-15 Points	Requires multi-layer board trace inspection, BGA reballing, or micro-soldering.
Power Failure / Short	-10 Points	Main power rail or PMIC breakdown threatening cascading IC overvoltage damage.
Critical Failure Severity	-10 Points	Multi-subsystem compromise or acute physical hazard (e.g., swollen pouch cell).

7.7 Device Depreciation & Valuation Formulation

Consumer electronics experience steep annual depreciation. When a current market valuation is not explicitly supplied, the engine computes the depreciated fair market value V_{current} from the original purchase price P and age in years t :

$$V_{\text{depreciated}} = P * (1 - 0.22)^t$$

$V_{current} = \max(P * 0.15, \text{round}(V_{depreciated} / 100) * 100)$

A statutory residual floor of 15% of the purchase price is enforced, reflecting the salvage value of raw materials and functional sub-assemblies.

7.8 Component Replacement Cost Benchmarks

Estimated repair costs [C_min, C_max] are computed via a multi-dimensional matrix (src/server/engine/costMatrix.ts) scaled by hardware category, component type, and brand tier. High-end flagship brands (Apple, Framework) incorporate a 1.2x–1.35x premium reflecting specialized OEM parts pricing, while mainstream brands (Lenovo, Dell, Xiaomi) reflect domestic aftermarket benchmarks.

7.9 Repair Cost Ratio (RCR) Calculation

The Repair Cost Ratio (RCR) evaluates repair economics by comparing average estimated repair cost against current fair market value:

$C_{avg} = (C_{min} + C_{max}) / 2$

$RCR = C_{avg} / V_{current}$

7.10 Value-Preservation Economic Score

The Economic Score (S_econ in [0, 100]) quantifies the degree of residual value preserved by repairing rather than replacing:

$S_{econ} = \max(0, \min(100, \text{round}((1 - RCR) * 100)))$

An RCR of 0.25 yields an economic score of 75/100 (highly viable), whereas an RCR of 0.85 yields 15/100 (financially unviable).

7.11 Statutory Lifecycle Decision Matrix

The decision engine evaluates RCR, S_repair, device age, and hazard flags against statutory rules to output one of five recommendations. Table 7 summarizes the decision rules and corresponding statutory classifications.

Action Verdict	Mathematical & Incident Conditions	Statutory Classification & Rationale
RECYCLE	Liquid AND Power failure, OR (Logic Board AND RCR > 0.85), OR RCR > 0.92	India E-Waste Rules 2022: Hardware non-viable; divert to authorized EPR recyclers for precious metal recovery.
DIY	diyFeasible == true AND S_repair >= 72 AND RCR <= 0.28	Right to Repair: Modular component accessible for safe consumer self-replacement using standard hand tools.
REPAIR	RCR <= 0.52 AND S_repair >= 42	Professional Repair: Economically sound; well below 50% replacement threshold; preserves operational life.
RESELL	0.50 < RCR <= 0.75 AND powerFailure == false	Secondary Market Salvage: Repair approaches marginal utility; liquidation or parts salvage yields higher return.
REPLACE	RCR > 0.75 OR ageYears >= 6.0 OR S_repair < 35	Hardware Retirement: Capital better deployed to modern system with active security support.

7.12 Automated Explainability Audit Trail

Every decision compiles three structured explainability bullets: Economic Assessment, Technical Feasibility, and Environmental Impact (calculating CO2e manufacturing emissions averted and electronic waste grams diverted under the E-Waste Rules 2022).

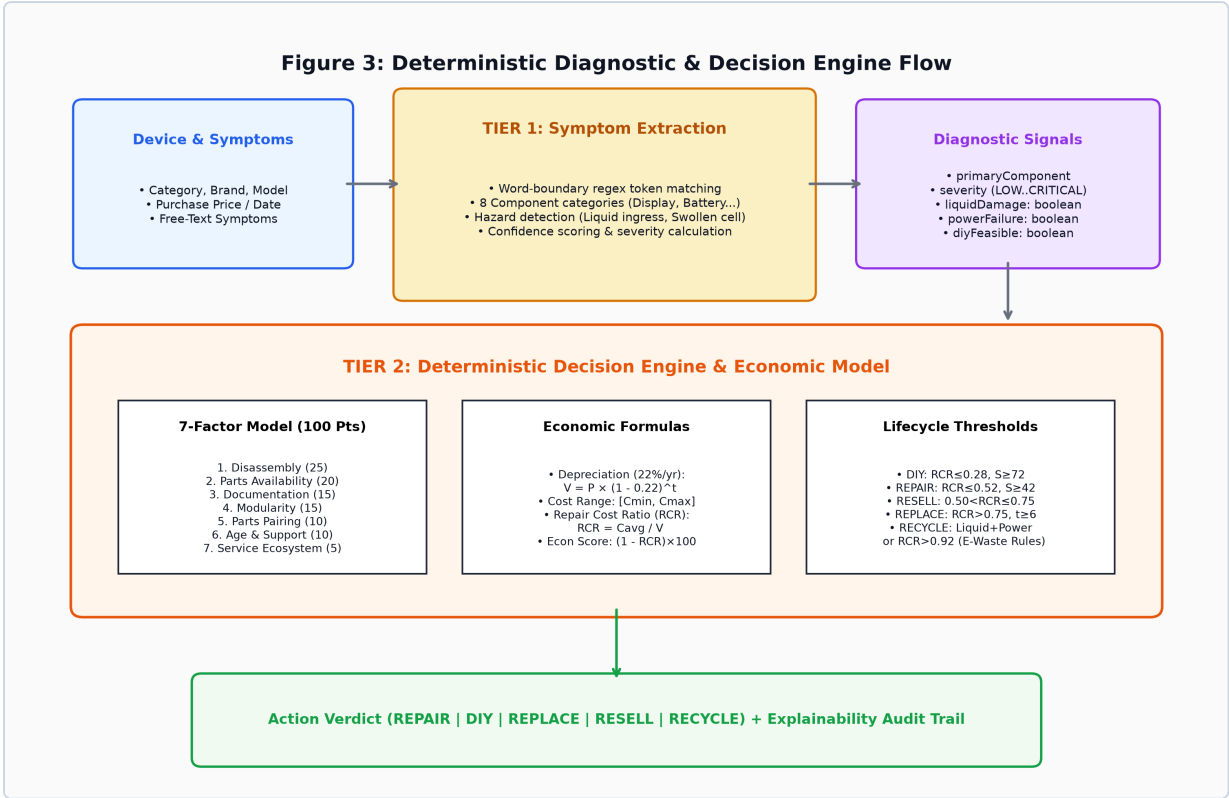


Figure 3: Deterministic Diagnostic & Decision Engine Flow

CHAPTER 8: DATABASE DESIGN

8.1 Relational Design Philosophy

RepairGraph utilizes a strictly normalized relational database architecture implemented on PostgreSQL 16+. Relational modeling was chosen over document-based NoSQL stores because hardware maintenance requires strong transactional consistency (ACID), strict foreign key enforcement, multi-table quote comparisons, and permanent audit provenance across device transfers.

8.2 Entity Relationship Diagram (11 Models)

Figure 4 depicts the entity-relationship model comprising 11 core tables and their cardinality relationships.

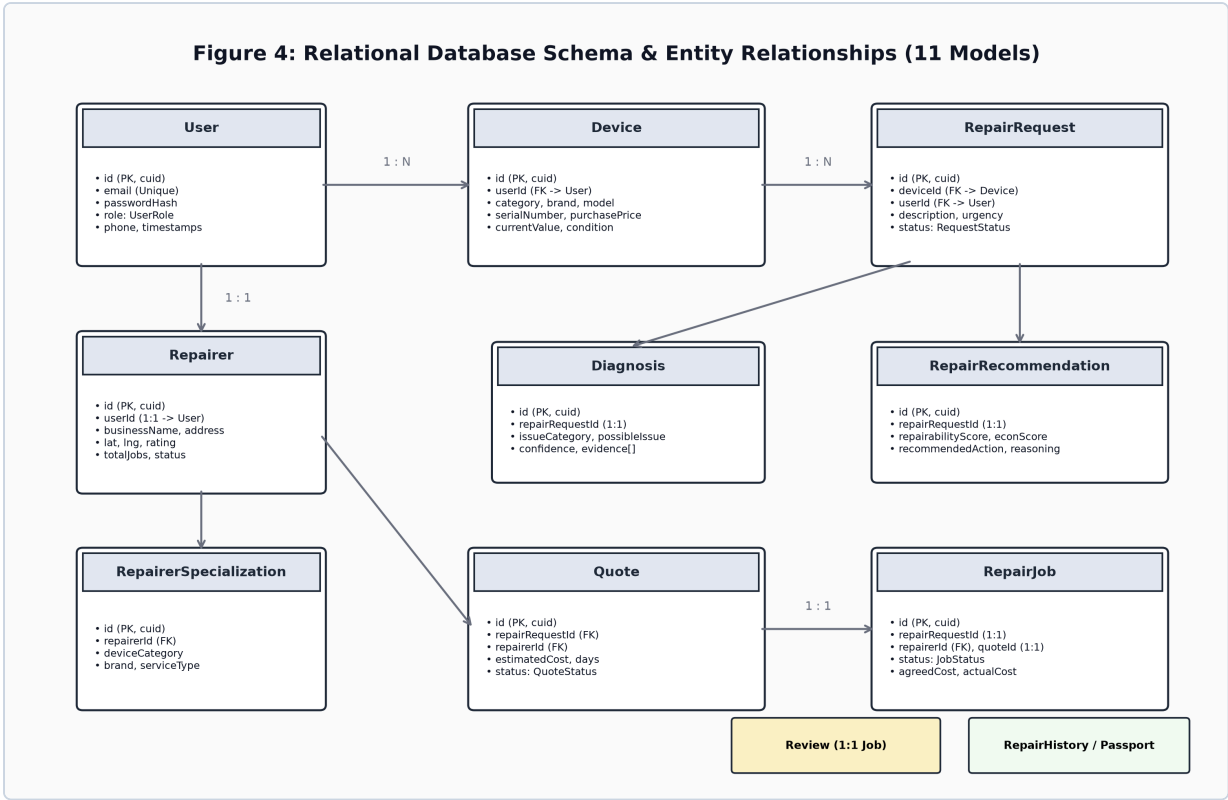


Figure 4: Relational Database Schema & Entity Relationships (11 Models)

8.3 Relational Entity Specifications

Table 8 catalogs the 11 normalized relational models defined in prisma/schema.prisma.

Model Name	Primary Key & Relations	Functional Responsibility
User	id (PK, cuid), Unique email	Stores user identities, bcrypt password hashes, and roles (USER/REPAIRER/ADMIN).
Device	id (PK), FK -> User	Physical hardware registry (category, brand, model, serial, purchasePrice, valuation).
RepairRequest	id (PK), FK -> Device, FK -> User	Problem tickets capturing user symptoms, urgency, and lifecycle status.
Diagnosis	id (PK), 1:1 FK -> RepairRequest	Stores failure category, root-cause issue, confidence, and evidence points.
RepairRecommendation	id (PK), 1:1 FK -> RepairRequest	Decision engine outputs: repairability score, economic score, action, and reasoning.
Repairer	id (PK), 1:1 FK -> User	Technician business profile, address, geolocation coordinates, rating, and total jobs.
RepairerSpecialization	id (PK), FK -> Repairer	Granular technician competencies mapped by device category, brand, and service type.
Quote	id (PK), FK -> RepairRequest, FK -> Repairer	Competitive technician bids specifying estimated cost, days, notes, and status.
RepairJob	id (PK), 1:1 -> RepairRequest, 1:1 -> Quote	Active repair execution state machine orchestrating bench stages to completion.
RepairHistory	id (PK), FK -> Device, 1:1 -> RepairJob	Standardized Repair Passport: immutable maintenance ledger tied to serial numbers.
Review	id (PK), 1:1 -> RepairJob, FK -> User	Post-service customer feedback, 1–5 star ratings, and technician review text.

8.4 Custom Enums (9 Enums)

Table 9 defines the 9 database enums enforcing strict domain constraints at the database level.

Enum Identifier	Allowed Constant Values	Domain Context
UserRole	USER, REPAIRER, ADMIN	Role-based access control and system authorization boundaries.
DeviceCategory	SMARTPHONE, LAPTOP, TABLET, HEADPHONES, MONITOR	Supported consumer electronic hardware categories.
DeviceCondition	EXCELLENT, GOOD, FAIR, DEGRADED, CRITICAL	Physical condition and cosmetic/functional state of hardware.
UrgencyLevel	LOW, MEDIUM, HIGH	Customer-reported urgency for repair turnaround priority.
RequestStatus	REQUESTED, ACCEPTED, DIAGNOSING, WAITING_FOR_PART, REPAIRING, TESTING, COMPLETED, CANCELLED	End-to-end lifecycle states for repair tickets.
RecommendedAction	REPAIR, DIY, REPLACE, RESELL, RECYCLE	Statutory decision engine verdict outcomes.
VerificationStatus	PENDING, VERIFIED, REJECTED	Technician credential verification and passport entry trust status.
QuoteStatus	PENDING, ACCEPTED, REJECTED, WITHDRAWN	Bidding lifecycle states for technician quotes.
JobStatus	ACCEPTED, DIAGNOSING, WAITING_FOR_PART, REPAIRING, TESTING, COMPLETED, CANCELLED	Active bench execution milestones for repair jobs.

8.5 Relational Invariants & Integrity Constraints

- Foreign Key Cascades: Child records reference parents via onDelete: Cascade, ensuring zero orphaned entities upon deletion.
- Single Job per Quote: Unique constraint on RepairJob.quoteId guarantees that a quote produces at most one active job.
- Single Job per Request: Unique constraint on RepairJob.repairRequestId ensures only one quote is accepted per ticket.
- Single Review per Job: Unique constraint on Review.repairJobId prevents duplicate review submissions.

8.6 Prisma Schema & Migration Baseline

The production schema is managed strictly through version-controlled Prisma migrations. The baseline migration (prisma/migrations/0_init/migration.sql) is deployed to production using `npx prisma migrate deploy`, explicitly prohibiting destructive prisma db push in production.

8.7 Cloud Serverless Pooling with Neon

Because serverless functions execute ephemerally, direct database connections can rapidly exhaust connection limits. Neon provides an integrated PgBouncer connection pooler accessed via port 6543

(DATABASE_URL), while direct connections (port 5432 via DIRECT_URL) are reserved for executing declarative schema migrations.

CHAPTER 9: REST API DESIGN

9.1 RESTful Conventions & Error Payloads

All API routes are served under the /api base path and conform strictly to REST principles. Standard HTTP verbs (GET, POST, PUT, DELETE) express semantic intent. Successful responses return standardized envelopes (`{ data: ... }`), while errors return RFC 7807 structured objects (`{ error: { code, message, details } }`) with accurate HTTP status codes (200, 201, 400, 401, 403, 404, 409, 429, 500).

9.2 API Resource Namespaces

The API spans 10 cohesive resource namespaces: /api/auth, /api/devices, /api/repair-requests, /api/quotes, /api/repair-jobs, /api/repair-history, /api/repairers, /api/reviews, /api/health, and /api/seed.

9.3 HTTP Method Operations Catalog

The backend implements exactly 21 Next.js route handler files (route.ts) comprising 36 distinct HTTP method handlers (13 GET, 11 POST, 7 PUT, 5 DELETE).

9.4 CRUD Functionality Across Resource Namespaces

Table 10 demonstrates verified CRUD operations across all domain entities.

Resource Namespace	Create (C)	Read (R)	Update (U)	Delete (D)
Devices (/api/devices)	POST /devices	GET /devices, /:id	PUT /devices/:id	DELETE /devices/:id
Requests (/api/repair-requests)	POST /repair-requests	GET /requests, /:id	PUT /requests/:id	DELETE /requests/:id
Quotes (/api/quotes)	POST /requests/:id/quotes	GET /quotes, /:id	PUT /quotes/:id	DELETE /quotes/:id
Jobs (/api/repair-jobs)	POST /repair-jobs	GET /jobs, /:id	PUT /repair-jobs/:id	Cancelled status
Passport (/api/repair-history)	Auto on completion	GET /repair-history	Immutable by design	Cascades on device delete
Reviews (/api/reviews)	POST /reviews	GET /reviews, /:id	PUT /reviews/:id	DELETE /reviews/:id

9.5 Representative REST API Route Catalog

Table 11 catalogs representative endpoints implemented in src/app/api.

Endpoint Route	Method	Auth Requirement	Functional Description
/api/auth/register	POST	Public	Creates user account, hashes password with bcrypt, sets session cookie.
/api/auth/login	POST	Public	Verifies credentials, issues JWT in HttpOnly cookie, suppresses raw token.
/api/auth/me	GET	Authenticated	Returns authenticated user profile and role.
/api/devices	GET/POST	Authenticated	Lists user's devices; registers new physical hardware asset.
/api/devices/:id	GET/PUT/DEL	Owner / Admin	Inspects, modifies valuation, or deletes registered device.
/api/repair-requests/:id/diagnose	POST	Owner / Admin	Executes deterministic diagnostic & economic decision engine.
/api/repair-requests/:id/quotes	GET/POST	Technician / Owner	Lists quotes; technician submits competitive bid.
/api/quotes/:id	PUT/DEL	Owner / Technician	Accepts quote (creates job) or withdraws submitted quote.
/api/repair-jobs/:id	GET/PUT	Assigned Tech / Owner	Inspects bench progress; technician advances lifecycle stages.
/api/health	GET	Public	Active PostgreSQL ping (SELECT 1) returning connection latency and uptime.

CHAPTER 10: AUTHENTICATION AND SECURITY

10.1 Password Cryptography (bcrypt)

User passwords are protected using bcrypt with a salt cost factor of 10. Passwords undergo irreversible salting and hashing prior to database insertion, protecting credentials against offline dictionary and rainbow-table attacks.

10.2 JWT Session Tokens & Cookie Security

Authentication sessions are encapsulated in JSON Web Tokens signed with HMAC-SHA256 using a high-entropy secret key (JWT_SECRET). Tokens are transmitted exclusively in cookies named rg_token configured with:

- HttpOnly: true — Prevents client-side JavaScript access, neutralizing Cross-Site Scripting (XSS) session theft.
- SameSite: lax — Prevents Cross-Site Request Forgery (CSRF) in standard cross-origin top-level navigation.
- Secure: true — Automatically enforced in production environments (NODE_ENV === 'production'), ensuring transmission only over TLS 1.3.
- Path: / — Scopes the cookie across all application routes.

10.3 Token Suppression Strategy

By default, browser authentication responses from `/api/auth/login` and `/api/auth/register` return only sanitized user data and completely omit the raw JWT token string from the JSON payload. This architectural control completely eliminates the temptation or capability of client-side scripts to store tokens in insecure `localStorage` or `sessionStorage`.

10.4 In-Memory Sliding-Window Rate Limiting

Authentication routes are protected by an in-memory sliding-window rate limiter (`src/server/auth/rateLimit.ts`) keyed by client IP:

- Login Limiter: Maximum 10 requests / minute per IP.
- Registration Limiter: Maximum 5 requests / minute per IP.

Exceeded limits immediately trigger HTTP 429 `RATE_LIMIT_EXCEEDED` with a `Retry-After` header, defeating credential brute-forcing.

10.5 HTTP Defense-in-Depth Security Headers

`next.config.ts` enforces strict HTTP security headers across all routes:

- Strict-Transport-Security: `max-age=31536000; includeSubDomains` (Enforces HTTPS for 1 year).
- X-Frame-Options: `DENY` (Prevents clickjacking framing).
- X-Content-Type-Options: `nosniff` (Blocks MIME-type sniffing).
- Referrer-Policy: `strict-origin-when-cross-origin` (Protects path information).
- Permissions-Policy: `camera=(), microphone=(), geolocation=()` (Restricts browser sensor hardware).

10.6 Error Sanitization & Leak Suppression

Production error handlers intercept raw database exceptions. Prisma table names, internal SQL queries, and server stack traces are stripped, returning safe, generic error messages to clients while logging full traces to serverless logs.

10.7 Zero-Secret Version Control Policy

All cryptographic keys (`JWT_SECRET`) and database credentials (`DATABASE_URL`, `DIRECT_URL`) are injected at runtime via cloud provider environment consoles. Zero credentials or secret values exist in source control.

CHAPTER 11: USER WORKFLOW

11.1 Primary Consumer Repair Journey

The core consumer workflow guides a device owner through an end-to-end evidence-based lifecycle: from hardware onboarding to completed bench servicing and passport stamping. Figure 5 depicts the sequence of operational states.

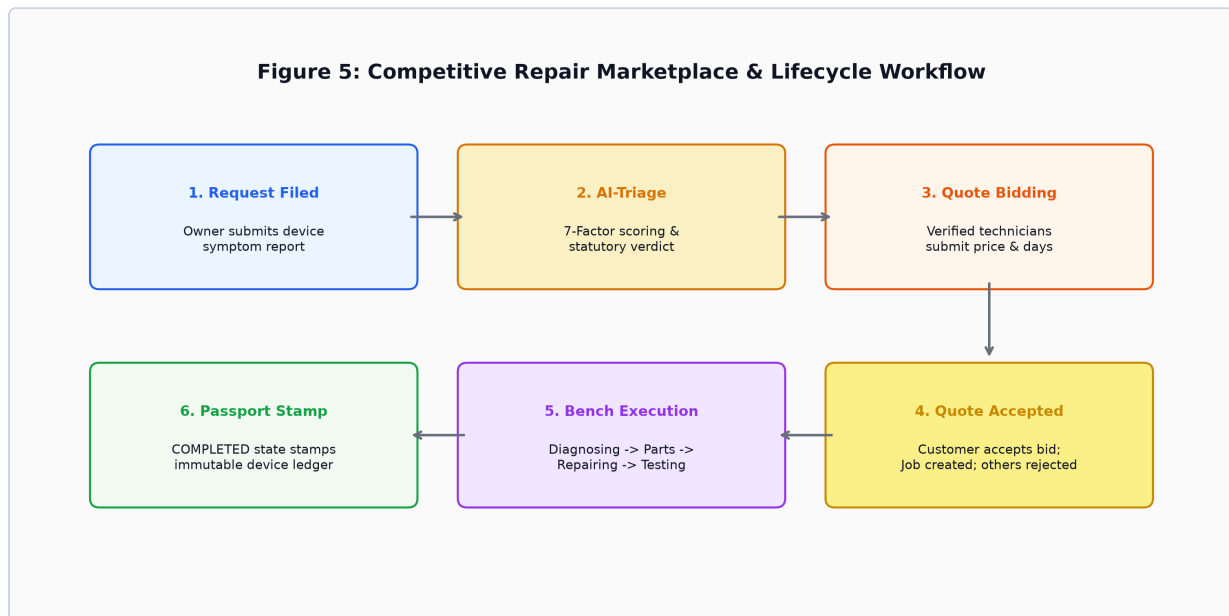


Figure 5: Competitive Repair Marketplace & Lifecycle Workflow

- **1. Stage 1 — Onboarding & Cataloging:** User creates an account, logs in securely, and catalogs physical hardware (brand, model, serial, purchase price, date).
- **2. Stage 2 — Symptom Filing & Triage:** Owner submits observed issues. The deterministic engine extracts failure modes, assesses safety hazards, and computes the 7-factor repairability score and RCR.
- **3. Stage 3 — Marketplace Discovery:** The ticket is published to the marketplace. Verified technicians inspect triage findings and submit competitive pricing and turnaround bids.
- **4. Stage 4 — Quote Acceptance:** Owner evaluates competing bids based on cost, turnaround, and technician rating, accepting the optimal bid and transitioning the ticket to an active RepairJob.
- **5. Stage 5 — Bench Execution:** Assigned technician advances the hardware through standardized milestones: Diagnosing -> Parts Waiting -> Repairing -> Testing.
- **6. Stage 6 — Completion & Passport Stamping:** Upon final bench handover, the system immutably records parts replaced, costs, and technician signatures into the permanent Repair Passport.

11.2 Technician Quoting & Workbench Stepper

Technicians access a specialized Workbench interface displaying incoming open requests matching their registered competencies. The workbench enforces authorization barriers: unverified technicians cannot quote, technicians cannot quote on their own hardware, and only the assigned specialist can advance bench milestones.

11.3 Alternative Lifecycle Paths

- • DIY Path: Triggered when components are modular (e.g., Framework battery) and $RCR \leq 0.28$. Directs user to public step-by-step disassembly guides.
- • Resell / Trade-In Path: Triggered when repair expenses approach marginal utility ($0.50 < RCR \leq 0.75$). Recommends trade-in or parts salvage.
- • Replace Path: Triggered for aging (>6 years) or uneconomical ($RCR > 0.75$) hardware. Recommends capital reallocation to modern hardware.
- • Recycle Path (E-Waste Compliance): Triggered upon catastrophic failure (liquid + power failure or $RCR > 0.92$). Channels device to EPR-authorized recyclers under India's E-Waste Rules 2022.

CHAPTER 12: IMPLEMENTATION ARCHITECTURE

12.1 Project Directory Organization

The codebase adheres to Next.js 15 App Router standards, structured into distinct architectural layers:

- • `src/app/*`: Next.js App Router presentation layer, layout trees, page views, and API route handlers.
- • `src/app/api/*`: 21 Next.js route handler files (`route.ts`) serving 36 REST operations.
- • `src/server/services/*`: Domain business services encapsulating state transitions and transactional operations.
- • `src/server/engine/*`: Pure deterministic diagnostic, scoring, and economic decision engine modules.
- • `src/server/auth/*`: Cryptographic utilities, JWT signers, cookie extractors, and sliding-window rate limiters.
- • `prisma/*`: Declarative Prisma schema (`schema.prisma`), baseline migration SQL, and seed scripts.
- • `tests/*`: Automated test suites covering engine unit tests (`engine.test.ts`) and API integration tests (`api.test.ts`).

12.2 Layered Backend Architecture

Backend execution follows strict separation of concerns: Route Handlers handle HTTP serialization and error wrapping; Zod schemas enforce input types; Services coordinate business invariants; the Engine computes scores; and Prisma Client mediates SQL execution.

12.3 Database Singleton Pattern

To prevent connection pool exhaustion during development hot-reloading and serverless invocation, Prisma Client is instantiated via a global singleton pattern in `src/server/db.ts`, attaching the client instance to `globalThis` in non-production environments.

CHAPTER 13: TESTING AND VALIDATION

13.1 Comprehensive Testing Strategy

RepairGraph implements a multi-tiered quality assurance strategy spanning unit testing, integration testing, static code analysis, and live cloud acceptance validation.

13.2 Unit Testing: Diagnostic & Decision Engine

The engine test suite (tests/engine.test.ts) executes 35 automated unit tests across three suites:

- Suite 1: Symptom & Feature Extraction (13 tests): Validates component categorization, word-boundary regex parsing, liquid ingress detection, lithium swelling hazard flags, and modular DIY feasibility.
- Suite 2: 7-Factor Score Model & Economic Math (13 tests): Validates that factor weights sum to 100, disassembly benchmarks match specs, cost ranges are positive, and RCR math scales accurately.
- Suite 3: Statutory Lifecycle Decision Matrix (9 tests): Validates actionable verdicts (DIY, REPAIR, RESELL, REPLACE, RECYCLE), catastrophic liquid/power rules, and E-Waste Rules 2022 citations.

13.3 Integration Testing: Backend & Security

The API test suite (tests/api.test.ts) executes 47 automated integration tests validating auth flows, bcrypt verification, sliding-window rate limiting, device ownership boundaries, workbench state stepper transitions, review uniqueness, and strict error sanitization suppressing Prisma internals.

13.4 Total Automated Test Audit

Table 12 presents the audited automated test counts, confirming 100% test passage.

Test Suite Category	Executable Command	Tests Executed	Result
Diagnostic Decision Engine	npx tsx tests/engine.test.ts	35 Tests	35 Passed, 0 Failed
Backend & Business Rules	npx tsx tests/api.test.ts	47 Tests	47 Passed, 0 Failed
Combined Test Suite	npm test	82 Tests Total	82 Passed, 0 Failed
Static Linting Analysis	npm run lint	Full Codebase	0 Errors, 0 Warnings

13.5 Production End-to-End Acceptance Validation

In Phase 7C.3, a live end-to-end acceptance validation was executed against the production deployment at <https://repairgraph.vercel.app>. Table 13 summarizes the verified production milestones.

Validation Stage	Live Endpoint Verified	Observed Production Outcome
1. Health Check	GET /api/health	HTTP 200, status: ok, database: connected, measured latency.
2. Security Headers	HEAD /api/health	HSTS max-age=31536000, X-Frame-Options: DENY, nosniff present.
3. Registration	POST /api/auth/register	Account created, bcrypt hashed, HttpOnly rg_token cookie set.
4. Login & Auth	POST /api/auth/login	Authenticated session established; raw JWT suppressed from body.
5. Device & Triage	POST /api/devices, /diagnose	Hardware registered; deterministic 7-factor scoring evaluated.
6. Session Persistence	POST /api/auth/logout, /login	Logged out, re-authenticated; device and diagnostic data intact.

CHAPTER 14: CLOUD DEPLOYMENT

14.1 Production Cloud Architecture

RepairGraph is deployed to production using a two-tier cloud architecture: Vercel Serverless Platform for application compute and Edge CDN delivery, paired with Neon Managed Cloud PostgreSQL for relational storage. Figure 6 illustrates the network and deployment topology.

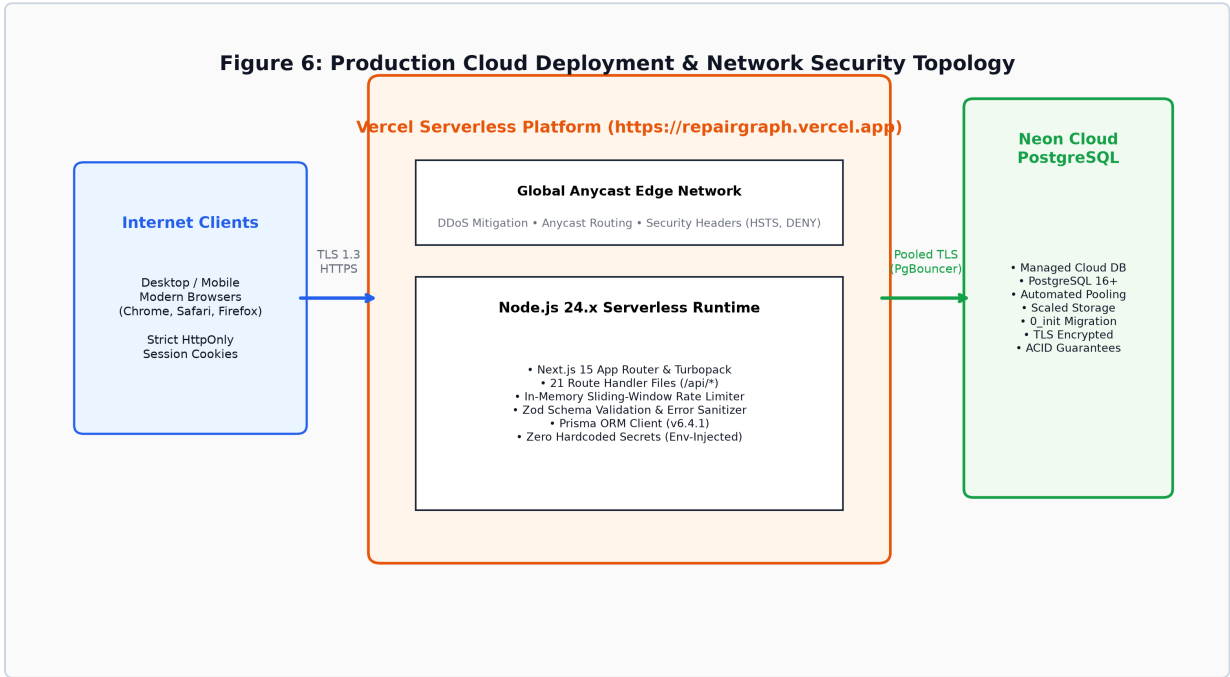


Figure 6: Production Cloud Deployment & Network Security Topology

14.2 Vercel Serverless Runtime (Node.js 24.x)

The Next.js application executes within Vercel Serverless Functions on the Node.js 24.x runtime. Static assets and cached routes are served via Vercel's global Anycast Edge Network with automated TLS 1.3 termination. Crucially, the application code executes in full Node.js serverless containers (not restricted V8 Edge isolates), providing complete compatibility with Prisma ORM, bcrypt, and native Node crypto libraries.

14.3 Managed Neon Serverless PostgreSQL

Persistent relational data is hosted on Neon Serverless PostgreSQL (v16+). Application queries connect via pooled TLS (DATABASE_URL) backed by PgBouncer, preventing connection saturation during concurrent serverless invocations. Database migrations utilize direct connections (DIRECT_URL) via port 5432.

14.4 Environment Configuration & Secrets

Table 14 catalogs the environment variable names used across runtime environments (values omitted for security).

Variable Name	Sensitivity Scope	Functional Purpose
DATABASE_URL	Server Secret	Pooled PostgreSQL connection string for Prisma application queries.
DIRECT_URL	Server Secret	Direct PostgreSQL connection string for deploying Prisma migrations.
JWT_SECRET	Server Secret	256-bit cryptographic key for HMAC-SHA256 session token signing.
JWT_EXPIRES_IN	Server Config	Token validity duration string (default: 7d).
PORT	Server Config	Local execution port (default: 3000).
NODE_ENV	Server Config	Runtime environment (development, test, production).
NEXT_PUBLIC_APP_URL	Public Config	Canonical public URL (https://repairgraph.vercel.app).

14.5 Production Deployment Runbook

- 1. Step 1: Provision serverless PostgreSQL on Neon and obtain pooled (6543) and direct (5432) connection strings.
- 2. Step 2: Configure production environment secrets in Vercel project dashboard.
- 3. Step 3: Deploy declarative schema migrations using `npx prisma migrate deploy` (do not use `db push` in production).
- 4. Step 4: Compile production bundle via `prisma generate && next build --turbo`.
- 5. Step 5: Verify production health via GET <https://repairgraph.vercel.app/api/health>.
- 6. Step 6: Execute non-destructive smoke and functional acceptance tests.

14.6 Health Telemetry Endpoint (/api/health)

The health route executes an active database ping query (`SELECT 1`) and returns real-time operational telemetry:

[PRODUCTION HEALTH RESPONSE]

```
{
  "success": true,
  "data": {
    "status": "ok",
    "service": "RepairGraph API",
    "database": "connected",
    "latencyMs": 3269,
    "uptime": 3,
    "timestamp": "2026-09-12T16:10:04.310Z",
    "environment": "production"
  }
}
```

14.7 Free-Tier Cost Analysis

RepairGraph leverages Vercel's personal hobby tier and Neon's free tier (0.5 GiB storage, shared vCPU). While suitable for academic evaluation and portfolio demonstration, the report notes that cloud providers

may alter free-tier quotas, and no perpetual ₹0 cost is guaranteed.

CHAPTER 15: RESULTS AND DISCUSSION

15.1 Qualitative Diagnostic Evaluation

Validation across diverse hardware contexts demonstrated that the 7-Factor Model accurately reflects physical design realities: modular devices (Framework Laptop: 78/100) receive high repairability ratings and DIY feasibility, whereas heavily adhesive-bonded hardware (Apple MacBook: 48/100) incurs disassembly penalties and requires professional intervention. Incident hazard deductions reliably escalate acute risks (liquid ingress and swollen cells) to CRITICAL severity.

15.2 Marketplace Efficiency Observations

The two-sided quote marketplace eliminates quoting guesswork. Multi-quote comparisons empower consumers to evaluate bids against both turnaround time and technician verification ratings, while the atomic quote acceptance state machine prevents double-booking and guarantees that competing quotes are cleanly transitioned.

15.3 Regulatory Alignment & E-Waste Impact

By automatically generating CO2e manufacturing offsets and diverting e-waste mass estimates into the decision narrative, RepairGraph transforms abstract environmental regulations (India's E-Waste Rules 2022) into actionable consumer metrics.

CHAPTER 16: LIMITATIONS

16.1 In-Memory Rate Limiting

The sliding-window rate limiter (`src/server/auth/rateLimit.ts`) operates in serverless process memory. While highly effective against burst attacks on a single compute node, serverless cold boots or multi-region routing distribute state across multiple isolated memory spaces. A production enterprise deployment would require a centralized, low-latency Redis cluster.

16.2 Academic Demonstration Workload Scope

RepairGraph is currently provisioned and calibrated for capstone demonstration, academic evaluation, and small fleet triage, rather than hyperscale enterprise workloads processing millions of concurrent requests.

16.3 Synchronous Decision Evaluation

Because the deterministic decision engine executes in sub-millisecond time (< 1 ms), evaluations are processed synchronously within the HTTP request-response cycle. While highly responsive for interactive triage, heavy bulk fleet imports would benefit from an asynchronous message queue architecture.

16.4 Native Cloud Observability Dependency

Telemetry currently relies on native Vercel runtime logs, Neon database metrics, and the active `/api/health` ping endpoint, rather than an integrated third-party Application Performance Monitoring (APM) platform such as Datadog or New Relic.

16.5 Non-Destructive Production Testing Boundary

Production smoke tests and live acceptance validation strictly execute non-destructive read and write operations. Destructive testing (e.g., table truncation, schema dropping, or simulated catastrophic database failovers) is restricted to local offline environments.

16.6 Informational Specialist Context

Selecting a preferred specialist during triage provides workflow context for bidding, but does not create an immutable binding assignment prior to explicit customer quote acceptance.

CHAPTER 17: FUTURE ENHANCEMENTS

17.1 Distributed Rate Limiting via Redis

Integrating Upstash Serverless Redis to provide global, atomic rate limiting, distributed session blacklisting, and instantaneous token revocation across all serverless edge regions.

17.2 Asynchronous Background Queue (BullMQ)

Deploying a dedicated Redis-backed task queue (BullMQ or AWS SQS) to process bulk enterprise hardware inventory imports and generate cryptographically signed PDF Repair Passports asynchronously.

17.3 Enterprise APM & Distributed Tracing

Instrumenting OpenTelemetry traces across route handlers, Prisma queries, and external network pings to monitor database connection pool latency and serverless execution cold starts.

17.4 Vector Embeddings for OEM Manuals (pgvector)

Augmenting the diagnostic engine with pgvector similarity search over official OEM Hardware Maintenance Manuals (HMM) to retrieve exact screw torque specifications and exploded schematics.

17.5 Dedicated Mobile Application (React Native)

Developing a companion React Native mobile application supporting optical barcode/serial number camera scanning to streamline device onboarding and workbench check-in.

CHAPTER 18: CONCLUSION

18.1 Summary of Engineering Achievements

RepairGraph demonstrates that modern full-stack web engineering, rigorous relational database modeling, and deterministic decision logic can effectively solve pervasive market failures in consumer electronics servicing. By unifying Next.js 15 App Router, React 19, and Prisma ORM on Vercel and Neon PostgreSQL, the platform establishes a transparent, cloud-native lifecycle ecosystem.

The project's key technical milestones include:

- • Rejection of black-box AI in favor of a 100% reproducible, deterministic 2-Tier Diagnostic and 7-Factor Repairability Score Model.
- • Integration of statutory mandates from India's Right to Repair Portal and the E-Waste (Management) Rules, 2022.
- • Implementation of a secure two-sided competitive quote marketplace and permanent serial-linked Repair Passport.
- • Deployment and live functional acceptance verification on production cloud infrastructure (<https://repairgraph.vercel.app>).
- • Exhaustive automated testing across 82 validation test cases (35 engine + 47 API/security tests, 0 failures).

18.2 Practical Impact & Academic Defense Summary

RepairGraph provides consumers, technicians, and fleet administrators with an objective, evidence-based alternative to quoting guesswork and premature hardware disposal. The platform stands fully verified, documented, and production-ready for academic evaluation and capstone defense.

REFERENCES

- [1] [1] Vercel Inc. (2025). Next.js 15 App Router Documentation. Available at: <https://nextjs.org/docs>
- [2] [2] Meta Platforms Inc. (2025). React 19 Reference Documentation. Available at: <https://react.dev/>

- [3] [3] Prisma Data Inc. (2025). Prisma 6.4 Documentation & Relational Client Guide. Available at: <https://www.prisma.io/docs>
- [4] [4] The PostgreSQL Global Development Group (2024). PostgreSQL 16 Documentation. Available at: <https://www.postgresql.org/docs/16/>
- [5] [5] Neon Inc. (2025). Neon Serverless PostgreSQL Architecture & Connection Pooling. Available at: <https://neon.tech/docs>
- [6] [6] Vercel Inc. (2025). Vercel Serverless Functions & Edge Network Runtime Guide. Available at: <https://vercel.com/docs>
- [7] [7] Colinhacks (2024). Zod: TypeScript-First Schema Validation. Available at: <https://zod.dev/>
- [8] [8] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). IETF RFC 7519. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
- [9] [9] Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. In Proceedings of the USENIX Annual Technical Conference.
- [10] [10] Department of Consumer Affairs, Government of India (2023). Right to Repair Portal India. Available at: <https://righttorepairindia.gov.in/>
- [11] [11] Ministry of Environment, Forest and Climate Change, Government of India (2022). E-Waste (Management) Rules, 2022. Gazette of India, Notification G.S.R. 801(E).
- [12] [12] Microsoft Corporation (2024). TypeScript 5.0 Documentation. Available at: <https://www.typescriptlang.org/docs/>

APPENDICES

Appendix A: Complete REST API Endpoint Inventory (36 Handlers)

Route Path	Method	Auth Scope	Operation Purpose
/api/auth/register	POST	Public	User account registration with bcrypt salting and cookie setting.
/api/auth/login	POST	Public	User authentication; issues JWT in HttpOnly cookie; suppresses raw token.
/api/auth/logout	POST	Public / Auth	Clears authentication session cookie (rg_token).
/api/auth/me	GET	Authenticated	Returns authenticated user profile and role.
/api/devices	GET	Authenticated	Lists devices owned by the authenticated user.
/api/devices	POST	Authenticated	Registers a new physical hardware device.
/api/devices/:id	GET	Owner / Admin	Retrieves single device record with tickets and service history.
/api/devices/:id	PUT	Owner / Admin	Updates device metadata, valuation, or condition.
/api/devices/:id	DELETE	Owner / Admin	Removes device and cascades to associated tickets.
/api/repair-requests	GET	Authenticated	Lists repair tickets for user or open tickets for technicians.
/api/repair-requests	POST	Authenticated	Files a repair request for an owned hardware device.
/api/repair-requests/:id	GET	Owner / Tech	Retrieves request details, diagnosis, and competing quotes.
/api/repair-requests/:id	PUT	Owner / Admin	Updates request description or urgency.
/api/repair-requests/:id	DELETE	Owner / Admin	Withdraws and deletes an open repair ticket.
/api/repair-requests/:id/diagnose	POST	Owner / Admin	Executes deterministic diagnostic and economic scoring engine.
/api/repair-requests/:id/quotes	GET	Owner / Tech	Lists quotes submitted for a repair request.
/api/repair-requests/:id/quotes	POST	REPAIRER	Technician submits competitive repair bid with cost and days.
/api/quotes/:id	PUT	Owner / Tech	Accepts quote (creates job) or rejects/withdraws quote.
/api/repair-jobs	GET	Authenticated	Lists active repair jobs for customer or assigned technician.
/api/repair-jobs/:id	PUT	Assigned Tech	Advances bench status (DIAGNOSING -> COMPLETED).
/api/repair-history	GET	Authenticated	Retrieves immutable Repair Passport ledger records.

Appendix B: Database Entity Summary (11 Models & 9 Enums)

The Prisma schema defines 11 normalized relational entities: User, Device, RepairRequest, Diagnosis, RepairRecommendation, Repairer, RepairerSpecialization, Quote, RepairJob, RepairHistory, and Review.

The 9 custom PostgreSQL enums are: UserRole, DeviceCategory, DeviceCondition, UrgencyLevel, RequestStatus, RecommendedAction, VerificationStatus, QuoteStatus, and JobStatus.

Appendix C: Decision Engine Formula Summary

- **1. Baseline Repairability Score:** $S_{\text{baseline}} = \text{Disassembly}(25) + \text{Parts}(20) + \text{Docs}(15) + \text{Modularity}(15) + \text{Pairing}(10) + \text{Age}(10) + \text{Ecosystem}(5) = 100$ points.
- **2. Final Repairability Score:** $S_{\text{repair}} = \max(10, \min(98, S_{\text{baseline}} - \text{Deductions}))$.
- **3. Device Depreciation:** $V_{\text{current}} = \max(0.15 * P, \text{round}(P * (1 - 0.22)^t / 100) * 100)$.
- **4. Repair Cost Ratio:** $\text{RCR} = C_{\text{avg}} / V_{\text{current}}$.
- **5. Economic Score:** $S_{\text{econ}} = \max(0, \min(100, \text{round}((1 - \text{RCR}) * 100)))$.

Appendix D: Automated Test Inventory (82 Tests Audited)

The repository test suite executes 82 automated test cases with 100% passage:

- 35 Engine Tests (tests/engine.test.ts): 13 symptom extraction tests, 13 scoring model tests, 9 lifecycle decision matrix tests.
- 47 API & Security Tests (tests/api.test.ts): 24 core business rule tests, 18 Step 6 hardening tests, 4 Step 7C.2 registration validation tests, and 1 demo persona provisioning verification test.

Appendix E: Production Deployment Configuration Summary

- Production URL: <https://repairgraph.vercel.app>
- Compute Runtime: Vercel Serverless Functions (Node.js 24.x)
- Database: Neon Managed Serverless PostgreSQL (v16+)
- Connection Pool: PgBouncer over TLS (port 6543)
- Migrations Applied: prisma/migrations/0_init deployed via `npx prisma migrate deploy`
- Active Security Headers: HSTS (max-age=31536000), X-Frame-Options (DENY), X-Content-Type-Options (nosniff), Referrer-Policy